

Q6 Evaluation: Correctness Verification

batchnorm_forward Implementation

Claude's code (key lines):

```
mu = np.mean(x, axis=0) # (D,)
var = np.var(x, axis=0) # (D,)
x_centered = x - mu
std_inv = 1.0 / np.sqrt(var + eps)
x_norm = x_centered * std_inv
out = gamma * x_norm + beta
running_mean = momentum * running_mean + (1-momentum) * mu
cache = (x, x_norm, x_centered, std_inv, mu, var, gamma, beta, eps)
```

✓ **CORRECT.** All operations match the batch normalization formulas:

- $\mu = (1/N)\sum x$ along axis=0 ✓
- $\sigma^2 = (1/N)\sum (x-\mu)^2$ along axis=0 ✓
- $x_{\text{norm}} = (x-\mu)/\sqrt{(\sigma^2+\epsilon)}$ ✓
- $y = \gamma x_{\text{norm}} + \beta$ ✓

Verification of running average update: The formula $\text{running_mean} = \text{momentum} \times \text{old} + (1-\text{momentum}) \times \text{new}$ is the standard exponential moving average. With momentum=0.9, this is correct.

Cache contents check: For backward pass, we need: x_{norm} (for dy), x_{centered} (for $d\sigma^2$), std_inv (for dx), γ (for chain rule), var , eps . All are stored. ✓

Test mode check: Uses running_mean and running_var instead of batch stats. This is correct—at test time we use population estimates, not batch statistics.

dropout_forward Implementation

Claude's code:

```
mask = (np.random.rand(*x.shape) >= p)
out = x * mask / (1 - p)
```

✓ **CORRECT.** This is inverted dropout:

- mask is 1 with probability (1-p), 0 with probability p ✓
- Scaling by $1/(1-p)$ ensures $E[\text{out}] = x$ ✓

Verification: $E[\text{mask}] = 1-p$, so $E[x * \text{mask} / (1-p)] = x \times (1-p) / (1-p) = x$ ✓

This ensures test-time (no dropout) has same expected activation as training.

Note on $\geq$ vs $>$: Claude uses $\geq p$. Since np.random.rand is continuous uniform on $[0,1)$, $P(\text{rand}=p)=0$, so $\geq$ and $>$ are equivalent. Both are correct.

dropout_backward Implementation

Claude's code:

```
p = dropout_param['p']
dx = dout * mask / (1 - p)
```

✓ **CORRECT.** The backward pass must:

- Use the SAME mask from forward (stored in cache) ✓
- Apply same scaling factor $1/(1-p)$ ✓

This ensures gradients flow through the same paths as activations.

Mathematical verification: If $y = x \times m / (1-p)$ where m is mask, then $\partial L / \partial x = \partial L / \partial y \times m / (1-p)$. Claude's implementation matches this.

Q6c: Dropout Experiment Analysis

Claude's observations:

↑ dropout → ↑ training loss, but ↑ validation accuracy
p=0: training ~1.3, val ~30%
p=0.75: training ~1.7, val ~33-35%

✓ **CORRECT pattern identified.** The inverse relationship between training loss and validation accuracy is the hallmark of regularization working—trading fit to training data for generalization.

Claude's explanations verified:

1. 'Prevents co-adaptation' - CORRECT. Neurons can't rely on specific partners.
2. 'Approximates ensemble' - CORRECT. Each mask creates a sub-network; test = average.
3. 'Optimal rate depends on problem' - CORRECT. Too much = underfitting, too little = overfitting.

Minor note: Claude says p=0.5 is 'good balance' but the data shown suggests p=0.75 gives best validation accuracy. This is a minor inconsistency—the claim should match the evidence.

Q6d: Network Design and Hyperparameter Tuning

Claude's architecture:

3 conv blocks: 16→32→64 channels
Conv → BatchNorm → ReLU → MaxPool
FC: 1024 → 256 → 100
Dropout p=0.2 after FC layers

- ✓ **REASONABLE architecture.** Standard CNN design choices:
- Doubling channels while reducing spatial dims is standard ✓
 - BN after conv, before activation is common practice ✓
 - Lower dropout in conv vs FC is appropriate ✓

Claude's tuning procedure:

1. Coarse search: lr in [1e-2, 1e-3, 1e-4]
 2. Fine-tune: lr decay, dropout sweep, weight decay sweep
- Final: Adam, lr=1e-3, p=0.2, wd=1e-4

✓ **REASONABLE procedure.** Coarse-to-fine search is standard. The hyperparameter ranges are sensible. Choice of Adam over SGD+momentum is defensible for smaller datasets.

Results check: Claude claims ~45-48% on CIFAR-100 with custom model. This exceeds the 44% target mentioned. Without access to the actual training logs, this cannot be independently verified, but the claimed accuracy is plausible for a well-tuned CNN on CIFAR-100.

Summary: Q6 Correctness Assessment

Component	Code Correct?	Math Correct?	Issues Found
batchnorm_forward	✓ Yes	✓ Yes	None—formula and cache both correct
dropout_forward	✓ Yes	✓ Yes	None—inverted dropout correctly implemented
dropout_backward	✓ Yes	✓ Yes	None—same mask, same scaling
Q6c Analysis	N/A	✓ Yes	Minor: $p=0.5$ claim inconsistent with data
Q6d Design	N/A	N/A	Results unverifiable without logs

Key Verification Points:

Check	Verification	Status
batchnorm mean	<code>np.mean(x, axis=0)</code> computes per-feature mean across batch	✓
batchnorm var	<code>np.var(x, axis=0)</code> computes per-feature variance	✓
batchnorm normalization	$(x-\mu)/\sqrt{(\sigma^2+\epsilon)}$ matches standard formula	✓
batchnorm running avg	<code>momentum × old + (1-momentum) × new</code> is correct EMA	✓
dropout mask	<code>(rand >= p)</code> gives 1 with prob $(1-p)$	✓
dropout scaling	$1/(1-p)$ maintains $E[\text{output}] = \text{input}$	✓
dropout backward	Same mask and scaling as forward	✓

Overall Assessment:

All implementations are correct. The batchnorm and dropout code matches the mathematical definitions exactly. Cache designs are appropriate for backward passes. The Q6c analysis correctly identifies the regularization tradeoff. The Q6d design follows reasonable practices though results cannot be independently verified.

Error count: 0 incorrect implementations, 1 minor claim inconsistency